

IMAGE AND VIDEO PROCESSING LAB

LABORATORY RECORD

FOR

M.TECH (DATA SCIENCE)

I YEAR, I SEMESTER



Submitted By

P. SWAMY

24011DB013

DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE AND TECHNOLOGY HYDERABAD

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

TELANGANA

(2024-2026)

UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE AND TECHNOLOGY HYDERABAD

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

Kukatpally, Hyderabad-500085



CERTIFICATE

This is to certify that the record entitled “**IMAGE AND VIDEO PROCESSING LAB**” is being submitted by **P. SWAMY**, Reg. No. **24011DB013** in partial fulfillment of the requirements for the award of the degree of **MASTER OF TECHNOLOGY** in the **DEPARTMENT OF INFORMATION TECHNOLOGY, UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE AND TECHNOLOGY HYDERABAD, JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD** is a record of bonafide work carried out by him under my guidance and supervision.

The results embodied in this record have not been submitted to any other university or institution for the award of any degree or diploma.

INTERNAL EXAMINER

EXTERNAL EXAMINER

LIST OF EXPERIMENTS

S.NO	NAME OF THE EXPERIMENT	PAGE NO.
1	Implement any four Image Enhancement Algorithms	1-2
2	Frequency Domain Filtering Algorithms	3-5
3	Image Restoration Algorithms	6-8
4	Color Image Processing Algorithms	9-10
5	Image compression techniques	11
6	Morphological Image Processing Methods	12-13
7	Image Segmentation Algorithms	14
8	Spatial Filtering Algorithms for image enhancement and segmentation	15-16
9	Object Recognition algorithms	17-19
10	Video restoration techniques	20-21
11	Video segmentation techniques	22-23
12	Video compression techniques	24-25

1.Implement any four image enhancement algorithms

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread('object.jpg', 0)
plt.subplot(3,3,1)
plt.imshow(image,cmap='gray')
plt.title('original image')
```

HISTOGRAM EQUALIZATION

```
def histogram_equalization(image):
    return cv2.equalizeHist(image)
equalized_image=histogram_equalization(image) plt.subplot(3,3,1)
plt.imshow(image,cmap='gray')
plt.title('Histogram equalization')
```

CONTRAST STRETCHING

```
def contrast_stretching(image):
    min_val=np.min(image)
    max_val=np.max(image)
    contrast_img=np.uint8(255*(image-min_val)/(max_val-min_val))
    return contrast_img
contrast_img=contrast_stretching(image) plt.subplot(3,3,2)
plt.imshow(image,cmap='gray')
plt.title('contrast stretching')
```

SMOOTHING

```
def smoothing(image,kernel_size=(5,5)):
    return cv2.GaussianBlur(image,kernel_size,0) smoothed_img=smoothing(image)
plt.subplot(3,3,3)
```

```
plt.imshow(image,cmap='gray') plt.title('smoothing')

# SHARPENING
def sharpening(image):
    kernel=np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
    return cv2.filter2D(image,-1,kernel) sharpened_img=sharpening(image)

plt.subplot(3,3,2)
plt.imshow(image,cmap='gray') plt.title('sharpening')
```

Output:



2. Frequency domain filtering algorithms

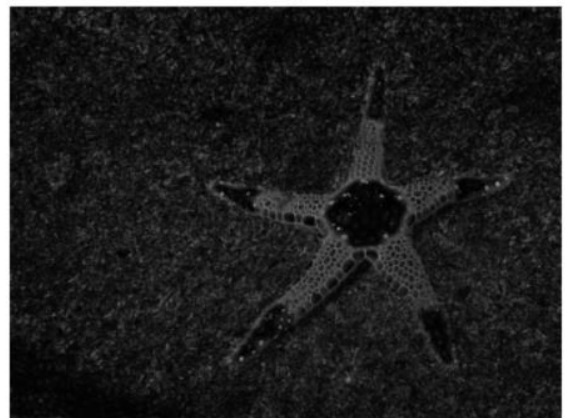
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image=cv2.imread('object.jpg', 0)
# HIGH PASS FILTER
f=np.fft.fft2(image)
fshift=np.fft.fftshift(np.fft.fft2(image))
rows,cols=image.shape
crow,ccol=rows//2,cols//2
radius=30
mask=np.ones((rows,cols),dtype=np.uint8)
cv2.circle(mask,(ccol,crow),radius,0,thickness=-1) fshift_filtered=fshift*mask
f_ishift=np.fft.ifftshift(fshift_filtered) image_filtered=np.fft.ifft2(f_ishift)
image_filtered=np.abs(image_filtered)
plt.figure(figsize=(10,5)) plt.subplot(1,2,1)
plt.title("original image")
plt.imshow(image,cmap='gray')
plt.subplot(1,2,2)
plt.title("enhanced image(high_pass)") plt.imshow(image_filtered,cmap='gray') plt.show()
```

Output:

Original Image



Enhanced Image (High Pass)



LOW PASS FILTER

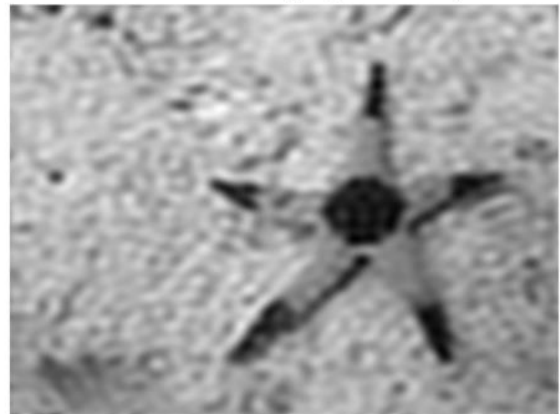
```
f=np.fft.fft2(image)
fshift=np.fft.fftshift(np.fft.fft2(image))
rows,cols=image.shape
crow,ccol=rows//2,cols//2
radius=30
mask_lp=np.zeros((rows,cols),dtype=np.uint8)
cv2.circle(mask_lp,(ccol,crow),radius,1,thickness=-1) fshift_filtered=fshift*mask_lp
f_ishift=np.fft.ifftshift(fshift_filtered) image_filtered=np.fft.ifft2(f_ishift)
image_filtered=np.abs(image_filtered)
plt.figure(figsize=(10,5)) plt.subplot(1,2,1)
plt.title("original image")
plt.imshow(image,cmap='gray')
plt.subplot(1,2,2)
plt.title("enhanced image(low_pass)") plt.imshow(image_filtered,cmap='gray') plt.show()
```

Output:

Original Image



Enhanced Image (Low Pass)



BANDPASS FILTER

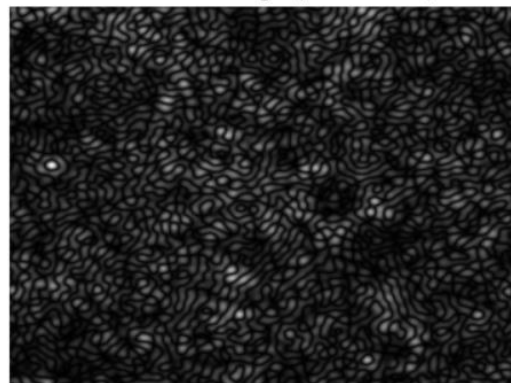
```
f=np.fft.fft2(image)
fshift=np.fft.fftshift(np.fft.fft2(image))
rows,cols=image.shape
crow,ccol=rows//2,cols//2
mask_bp=np.zeros((rows,cols),dtype=np.uint8) low_radius=20
high_radius=40
cv2.circle(mask_bp, (ccol, crow), high_radius, 1, thickness=-1) cv2.circle(mask_bp, (ccol, crow),
low_radius, 0, thickness=-1)
fshift_filtered=fshift*mask_bp
f_ishift=np.fft.ifftshift(fshift_filtered) image_filtered=np.fft.ifft2(f_ishift)
image_filtered=np.abs(image_filtered)
plt.figure(figsize=(10,5)) plt.subplot(1,2,1)
plt.title("original image")
plt.imshow(image,cmap='gray')
plt.subplot(1,2,2)
plt.title("enhanced image(band_pass)")
plt.imshow(image_filtered,cmap='gray') plt.show()
```

Output:

Original Image



Enhanced Image (Band Pass)



3. Image Restoration algorithms

WIENER FILTER

```
import numpy as np
import cv2
from scipy.signal import convolve2d
import matplotlib.pyplot as plt

def wiener_filter(image, kernel, noise_var, image_var):
    kernel_ft = np.fft.fft2(kernel, s=image.shape)
    image_ft = np.fft.fft2(image)
    wiener_filter = np.conj(kernel_ft) / (np.abs(kernel_ft)**2 + noise_var/image_var)
    restored_ft = wiener_filter * image_ft
    restored_image = np.abs(np.fft.ifft2(restored_ft))
    return restored_image

image = cv2.imread('object.jpg', 0)
kernel = np.ones((5, 5)) / 25 # Simple averaging filter
noise_var = 0.1 # Noise variance (adjust as necessary)
image_var = 1 # Image variance (adjust as necessary)
restored_image = wiener_filter(image, kernel, noise_var, image_var)
plt.imshow(restored_image, cmap='gray')
plt.title('Restored Image using Wiener Filter')
plt.show()
```

Output:

Original Image



Restored Image (Wiener Filter)



SALT AND PAPPER:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread('object.jpg', 0)
def salt_and_pepper_noise(image, amount=0.02):
    noisy_image = image.copy()
    total_pixels = noisy_image.size
    num_salt = int(total_pixels * amount * 0.5)
    num_pepper = int(total_pixels * amount * 0.5)
    salt_coords = [np.random.randint(0, i - 1, num_salt) for i in noisy_image.shape]
    noisy_image[salt_coords[0], salt_coords[1]] = 255
    pepper_coords = [np.random.randint(0, i - 1, num_pepper) for i in noisy_image.shape]
    noisy_image[pepper_coords[0], pepper_coords[1]] = 0
    return noisy_image
noisy_image_sp = salt_and_pepper_noise(image)
restored_image_sp = cv2.medianBlur(noisy_image_sp, 3)
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(noisy_image_sp, cmap='gray')
plt.title('Noisy Image (Salt and Pepper)')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(restored_image_sp, cmap='gray')
plt.title('Restored Image (Median Filter)')
plt.axis('off')
plt.tight_layout()
plt.show()
```

OUTPUT:

Original Image



Noisy Image (Salt & Pepper)



Restored Image (Median Filter)



4. Color image processing algorithms

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# RGB TO GRAY SCALE
image = cv2.imread('object.jpg') gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)) plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(gray_image,cmap='gray') plt.title("Grayscale image")
plt.axis('off') plt.show()
```

output:



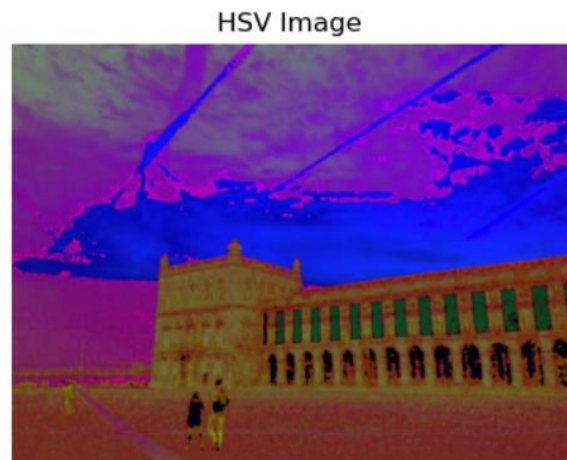
RGB-HSV

```
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)) plt.title("Original Image")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(hsv_image) plt.title('HSV Image') plt.axis('off')
```

```
plt.show()
```

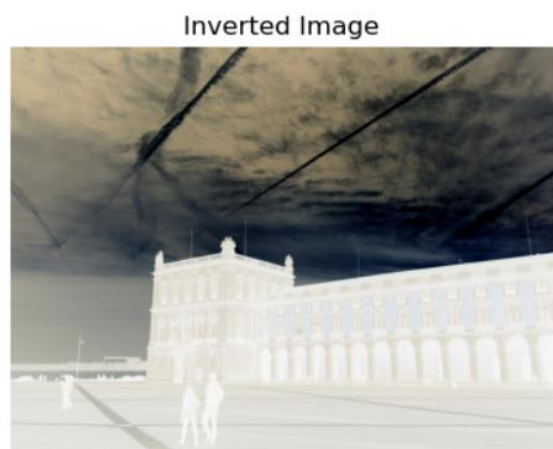
output:



COLOR INVERSION

```
inverted_image = cv2.bitwise_not(image) plt.figure(figsize=(10, 5))  
plt.subplot(1, 2, 1)  
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)) plt.title("Original Image")  
plt.axis('off')  
plt.subplot(1,2,2)  
plt.imshow(cv2.cvtColor(inverted_image, cv2.COLOR_BGR2RGB)) plt.title('Inverted  
Image')  
plt.axis('off') plt.show()
```

Output:



5. Image compression techniques

```
import cv2
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

def jpeg_compression(image_path, output_path, quality=50):
    image = cv2.imread(image_path)
    cv2.imwrite(output_path, image, [cv2.IMWRITE_JPEG_QUALITY, quality])
    return image

def png_compression(image_path, output_path, compression_level=9):
    image = Image.open(image_path)
    image.save(output_path, 'PNG', compress_level=compression_level)
    return np.array(image)

def plot_images(original, jpeg, png):
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    for ax, img, title in zip(axes, [original, jpeg, png], ["Original Image", "JPEG Compressed",
    "PNG Compressed"]):
        ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB) if len(img.shape) == 3 else img)
        ax.set_title(title)
        ax.axis("off")
    plt.show()

if __name__ == "__main__":
    input_image = "input.jpg"
    original = cv2.imread(input_image)
    jpeg_image = jpeg_compression(input_image, "compressed.jpg", quality=30)
    png_image = png_compression(input_image, "compressed.png", compression_level=5)
    plot_images(original, jpeg_image, png_image)
```



6. Morphological Image Processing Methods

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

image = cv2.imread('object.jpg', cv2.IMREAD_GRAYSCALE)

kernel = np.ones((5, 5), np.uint8)

# APPLY MORPHOLOGICAL OPERATIONS

dilated = cv2.dilate(image, kernel, iterations=1)

eroded = cv2.erode(image, kernel, iterations=1)

opened = cv2.morphologyEx(image, cv2.MORPH_OPEN, kernel)

closed = cv2.morphologyEx(image, cv2.MORPH_CLOSE, kernel)

# Plot results

plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)

plt.imshow(image, cmap='gray')

plt.title('Original Image')

plt.axis('off')

plt.subplot(2, 3, 2)

plt.imshow(dilated, cmap='gray')

plt.title('Dilated Image')

plt.axis('off')

plt.subplot(2, 3, 3)

plt.imshow(eroded, cmap='gray')

plt.title('Eroded Image')

plt.axis('off')

plt.subplot(2, 3, 4)

plt.imshow(opened, cmap='gray')

plt.title('Opened Image')

plt.axis('off')

plt.subplot(2, 3, 5)
```

```
plt.imshow(closed, cmap='gray')  
plt.title('Closed Image')  
plt.axis('off')  
plt.tight_layout()  
plt.show()
```

Output:

Original Image



Dilated Image



Eroded Image



Opened Image



Closed Image



7. Image Segmentation Algorithms

THRESHOLDING AND EDGE DETECTION

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

image = cv2.imread('object.jpg', cv2.IMREAD_GRAYSCALE)

# APPLY THRESHOLDING

_, thresholded_image = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

# APPLY THE CANNY EDGE DETECTION

edges = cv2.Canny(image, 100, 200)

# Display both images side by side

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)

plt.imshow(thresholded_image, cmap='gray')

plt.title("Thresholded Image")

plt.axis('off')

plt.subplot(1, 2, 2)

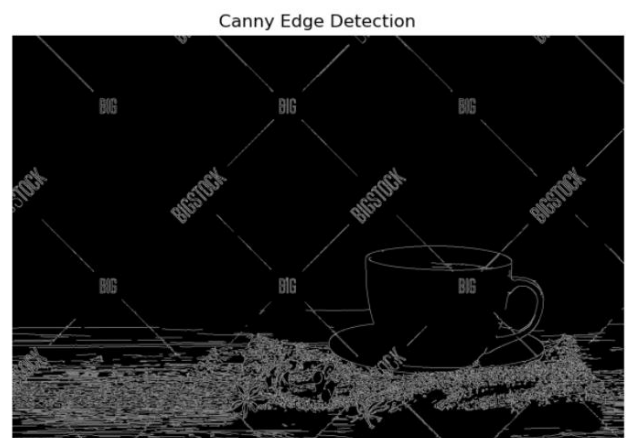
plt.imshow(edges, cmap='gray')

plt.title("Canny Edge Detection Image")

plt.axis('off')

plt.tight_layout()

plt.show()
```



8. Spatial Filtering Algorithms for Image Enhancement and Segmentation

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Load an image

image = cv2.imread('object.jpg', cv2.IMREAD_GRAYSCALE)


# DEFINE SPATIAL FILTERS (KERNELS)

sharpen_kernel = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]])

blur_kernel = np.ones((5, 5), np.float32) / 25 # Average filter

edge_detection_kernel = np.array([[ -1, -1, -1],
                                   [-1, 8, -1],
                                   [-1, -1, -1]])


# APPLY FILTERS USING CV2.FILTER2D

sharpened = cv2.filter2D(image, -1, sharpen_kernel)

blurred = cv2.filter2D(image, -1, blur_kernel)

edges = cv2.filter2D(image, -1, edge_detection_kernel)

# Display results

titles = ['Original Image', 'Sharpened', 'Blurred', 'Edge Detection']

images = [image, sharpened, blurred, edges]


plt.figure(figsize=(12, 6))

for i in range(4):

    plt.subplot(2, 2, i+1)

    plt.imshow(images[i], cmap='gray')
```

```
plt.title(titles[i])  
plt.axis('off')  
plt.show()
```

Output:

Original Color Image



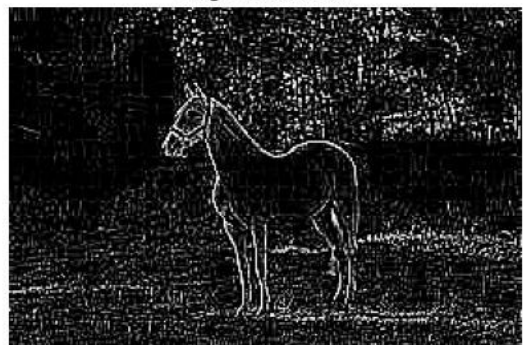
Sharpened



Blurred



Edge Detection



9. Object Recognition algorithms

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image and the template
image = cv2.imread('object.jpg')
template = cv2.imread('object.jpg')

# Ensure the images are loaded correctly
if image is None or template is None:
    print("Error: Could not load images. Check the file paths.")
    exit()

# Convert images to grayscale
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)

# Ensure the template is smaller than the image
if template_gray.shape[0] > image_gray.shape[0] or template_gray.shape[1] > image_gray.shape[1]:
    scale_percent = 50 # Reduce template size by 50% if needed
    width = int(template_gray.shape[1] * scale_percent / 100)
    height = int(template_gray.shape[0] * scale_percent / 100)
    template_gray = cv2.resize(template_gray, (width, height), interpolation=cv2.INTER_AREA)

# Perform template matching
result = cv2.matchTemplate(image_gray, template_gray, cv2.TM_CCOEFF_NORMED)

# Get the best match location
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)
top_left = max_loc
```

```

h, w = template_gray.shape

# Draw a thick rectangle around the matched area
bottom_right = (top_left[0] + w, top_left[1] + h)
cv2.rectangle(image, top_left, bottom_right, (0, 255, 0), 5) # Thickness = 5

# Display the result
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Template Matching Result (Thicker Border)")
plt.axis('off')
plt.show()

```

FEATURE-BASED OBJECT RECOGNITION USING ORB

```

import cv2

import matplotlib.pyplot as plt

# Load the image
image = cv2.imread('object.jpg', 0)

# ORB feature detector
orb = cv2.ORB_create()

# Detect keypoints
keypoints = orb.detect(image, None)

image_with_keypoints = cv2.drawKeypoints(image, keypoints, None, color=(0,255,0))

# Show result
plt.imshow(image_with_keypoints, cmap="gray")
plt.title("ORB Feature Detection")
plt.axis("off")
plt.show()

```

Output:



ORB Feature Detection



10. Video Restoration Techniques

DE- BLURRING: MOTION BLUR REMOVAL USING WIENER FILTER

```
import cv2

import numpy as np

def wiener_filter(img, kernel_size=5):

    # Apply Wiener Filter for motion blur restoration

    kernel = np.ones((kernel_size, kernel_size)) / kernel_size**2 # Simple motion blur kernel

    return cv2.filter2D(img, -1, kernel)

# Open video

cap = cv2.VideoCapture('video.mp4') # Replace with your video path

# Prepare the video writer to save the output

frame_height, frame_width = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)),
int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

out = cv2.VideoWriter('deblurred_video.mp4', cv2.VideoWriter_fourcc(*'mp4v'), 30, (frame_width,
frame_height))

while(cap.isOpened()):

    ret, frame = cap.read()

    if not ret:

        break

    # Apply Wiener filter (motion blur restoration)

    restored_frame = wiener_filter(frame)

    # Write the restored frame to the output video

    out.write(restored_frame)

cap.release()

out.release()

cv2.destroyAllWindows()
```

NOISE REDUCTION (DENOISING):

```
import cv2

# Open video

cap = cv2.VideoCapture('video.mp4') # Replace with your video path

# Prepare the video writer to save the output

frame_height, frame_width = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)),
int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

out = cv2.VideoWriter('denoised_video.mp4', cv2.VideoWriter_fourcc(*'mp4v'), 30, (frame_width,
frame_height))

while(cap.isOpened()):

    ret, frame = cap.read()

    if not ret:

        break

    # Denoising using Non-Local Means

    denoised_frame = cv2.fastNlMeansDenoisingColored(frame, None, 10, 10, 7, 21)

    # Write the denoised frame to the output video

    out.write(denoised_frame)

cap.release()

out.release()

cv2.destroyAllWindows()
```


11. Video Segmentation Techniques

BACKGROUND SUBTRACTION (FOR MOVING OBJECT SEGMENTATION):

```
import cv2

import numpy as np

# Open video

cap = cv2.VideoCapture('video.mp4') # Replace with your video path

# Create Background Subtractor

fgbg = cv2.createBackgroundSubtractorMOG2()

while(cap.isOpened()):

    ret, frame = cap.read()

    if not ret:

        break

    # Apply background subtraction

    fgmask = fgbg.apply(frame)

    # The result is the mask of the moving objects

    segmented_frame = cv2.bitwise_and(frame, frame, mask=fgmask)

    # Display the segmented frame

    cv2.imshow('Segmented Frame', segmented_frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):

        break

cap.release()

cv2.destroyAllWindows()
```

SEMANTIC VIDEO SEGMENTATION(PIXEL-LEVEL CLASSIFICATION):

```
import cv2

import numpy as np

# Load pre-trained DeepLabV3 model from OpenCV (MobileNetV2)

net = cv2.dnn.readNetFromTensorflow('deeplabv3_mnv2_tf_dim_ordering_tf_kernels.pb')

# Open video
```

```

cap = cv2.VideoCapture('video.mp4') # Replace with your video path
while(cap.isOpened()):
    ret, frame = cap.read()
    if not ret:
        break
    # Prepare frame for deep learning model (resize and normalize)
    blob = cv2.dnn.blobFromImage(frame, 1.0, (512, 512), (127.5, 127.5, 127.5), swapRB=True,
crop=False)
    net.setInput(blob)
    output = net.forward() # Perform forward pass to get the segmentation
    output_mask = np.argmax(output[0], axis=0) # Get the segmentation mask
    # Map the mask to colors (here just using a simple color map)
    segmented_frame = cv2.applyColorMap(output_mask.astype(np.uint8), cv2.COLORMAP_JET)
    # Show segmented frame
    cv2.imshow('Segmented Frame', segmented_frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

12. Video Compression Techniques

VIDEO COMPRESSION USING OPENCV (H.264 CODEC):

```
import cv2

# Open video
cap = cv2.VideoCapture('input_video.mp4')

# Get video properties
frame_width = int(cap.get(3))
frame_height = int(cap.get(4))
fps = cap.get(cv2.CAP_PROP_FPS)

# Define codec and create VideoWriter object
fourcc = cv2.VideoWriter_fourcc(*'X264') # H.264 codec
out = cv2.VideoWriter('output_video.mp4', fourcc, fps, (frame_width, frame_height))

while(cap.isOpened()):
    ret, frame = cap.read()
    if not ret:
        break

    # Write the frame to the output file
    out.write(frame)

cap.release()
out.release()
cv2.destroyAllWindows()
```

COMPRESSING VIDEO WITH AUDIO (USING FFMPEG COMMAND-LINE):

```
import cv2

# Open the input video
input_file = 'input_video.mp4'
cap = cv2.VideoCapture(input_file)

# Get the frame properties
frame_width = int(cap.get(3)) # Frame width
frame_height = int(cap.get(4)) # Frame height
fps = cap.get(cv2.CAP_PROP_FPS) # Frames per second

# Define the codec and create a VideoWriter object
fourcc = cv2.VideoWriter_fourcc(*'X264') # Using H.264 codec for video
out = cv2.VideoWriter('output_video.mp4', fourcc, fps, (frame_width, frame_height))

# Loop through each frame of the video
while(cap.isOpened()):
    ret, frame = cap.read()
    if not ret:
        break
    # Write each frame to the output file
    out.write(frame)
# Release the video capture and writer objects
cap.release()
out.release()
cv2.destroyAllWindows()
```

